1. Create a file named notes.txt and display its permissions string using the ls -l command.

```
akansha@localhost:~$ touch notes.txt
akansha@localhost:~$ ls -l notes.txt
-rw-r--r--. 1 akansha akansha 0 Oct 31 10:15 notes.txt
```

2. Change the permissions of notes.txt to 644 using numeric (octal) mode, and verify the changes using ls -l.

```
akansha@localhost:~$ chmod 644 notes.txt
akansha@localhost:~$ ls -l notes.txt
-rw-r--r--. 1 akansha akansha 0 Oct 31 10:15 notes.txt
```

3. Change notes.txt permissions to 600 (owner read/write only, no permissions for group and others) and verify.

```
akansha@localhost:~$ chmod 600 notes.txt
akansha@localhost:~$ ls -l notes.txt
-rw-------. 1 akansha akansha 0 Oct 31 10:16 notes.txt
```

4. Create an executable file named script.sh and grant execute permission exclusively to the owner using symbolic mode.

```
akansha@localhost:~$ touch script.sh
akansha@localhost:~$ chmod u+x script.sh
akansha@localhost:~$ ls -l script.sh
-rwx------. 1 akansha akansha 0 Oct 31 10:17 script.sh
```

5. Use the whoami command and ls -l notes.txt to identify the user and group ownership of notes.txt.

```
akansha@localhost:~$ whoami
akansha
akansha@localhost:~$ ls -l notes.txt
-rw-------. 1 akansha akansha 0 Oct 31 10:16 notes.txt
```

6. Run the ps command in your terminal and record how many processes are active in your current shell session.

```
akansha@localhost:~$ ps
    PID TTY          TIME CMD
   3241 pts/0    00:00:00 bash
   3512 pts/0    00:00:00 ps
```

7. Run ps aux and count or estimate the total number of running processes across the entire operating system.

```
akansha@localhost:~$ ps aux | wc -l
184
```

8. Start a background process using sleep 60 & and immediately run jobs to verify that it is active in the background.

```
akansha@localhost:~$ sleep 60 &
[1] 3548
akansha@localhost:~$ jobs
[1]+  Running                 sleep 60 &
```

9. Use the kill command with the Process ID (PID) to terminate the active sleep process created in Exercise 8.

```
akansha@localhost:~$ kill 3548
[1]+  Terminated              sleep 60 &
```

10. Search for an available software package named tree using your system's package manager without installing it.

```
akansha@localhost:~$ sudo dnf search tree
====================== Name Exactly Matched: tree ======================
tree.x86_64 : File system tree viewer
```

11. Display the current umask value of your terminal shell environment.

```
akansha@localhost:~$ umask
0022
```

12. Use symbolic chmod syntax to remove write permissions from the group on a file named document.txt.

```
akansha@localhost:~$ touch document.txt
akansha@localhost:~$ chmod g-w document.txt
akansha@localhost:~$ ls -l document.txt
-rw-r--r--. 1 akansha akansha 0 Oct 31 10:20 document.txt
```

13. Bring a background job with Job ID 1 back to the foreground using the fg command.

```
akansha@localhost:~$ sleep 100 &
[1] 3610
akansha@localhost:~$ fg %1
sleep 100
```

14. Send a background job with Job ID 1 to resume execution in the background using the bg command.

```
akansha@localhost:~$ bg %1
[1]+ sleep 100 &
```

15. Display the kernel information and system architecture using uname -a.

```
akansha@localhost:~$ uname -a
Linux rhel-host 5.14.0-284.11.1.el9_2.x86_64 #1 SMP PREEMPT_DYNAMIC x86_64
x86_64 x86_64 GNU/Linux
```

16. Display a snapshot of all processes running for your user account using ps -u $USER.

```
akansha@localhost:~$ ps -u $USER
    PID TTY          TIME CMD
   3241 pts/0    00:00:00 bash
   3610 pts/0    00:00:00 sleep
   3650 pts/0    00:00:00 ps
```

17. Use pkill to terminate a test process running under the name sleep.

```
akansha@localhost:~$ pkill sleep
[1]+  Terminated              sleep 100
```

18. Check current repository package updates on a Red Hat (RHEL) system using sudo dnf check-update (or apt update on Ubuntu).

```
akansha@localhost:~$ sudo dnf check-update
Updating Subscription Management repositories.
Metadata cache created.
```

19. Display the list of currently installed packages on your system using dnf list installed.

```
akansha@localhost:~$ dnf list installed | head -n 5
Installed Packages
audit.x86_64                                  3.0.7-103.el9
@anaconda
bash.x86_64                                   5.1.8-6.el9
@anaconda
coreutils.x86_64                              8.32-32.el9
@anaconda
```

20. Use chown to change the owner of script.sh to yourself explicitly, and verify with ls -l.

```
akansha@localhost:~$ sudo chown akansha script.sh
akansha@localhost:~$ ls -l script.sh
-rwx------. 1 akansha akansha 0 Oct 31 10:17 script.sh
```

21. Set permissions on script.sh so owner has full access (rwx), group has read/execute (r-x), and others zero permissions.

```
akansha@localhost:~$ chmod 750 script.sh
akansha@localhost:~$ ls -l script.sh
-rwxr-x---. 1 akansha akansha 0 Oct 31 10:17 script.sh
```

22. Explain the exact difference in rights between running chmod 700 file and running chmod 007 file.

```
Explanation:
- chmod 700: Owner gets full permissions (rwx), while Group and Others have no
access (---).
- chmod 007: Owner and Group have no access (---), while Others get full
permissions (rwx).
```

23. Use chown to change both the user owner and group owner of a file to akansha:akansha in a single command.

```
akansha@localhost:~$ sudo chown akansha:akansha notes.txt
akansha@localhost:~$ ls -l notes.txt
-rw-------. 1 akansha akansha 0 Oct 31 10:16 notes.txt
```

24. Launch the top interactive process monitor, press 'P' to sort processes by CPU utilization, and press 'q' to exit.

```
akansha@localhost:~$ top
top - 10:25:11 up  2:14,  1 user,  load average: 0.05, 0.03, 0.00
Tasks: 185 total,   1 running, 184 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.3 us,  0.2 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 3241 akansha   20   0  225412   5120   3840 S   0.3   0.2   0:00.12 bash
    1 root      20   0  168432  13450   8920 S   0.0   0.5   0:01.85 systemd
```

25. Start two background sleep processes (sleep 100 & and sleep 200 &), list them with jobs, and bring job 1 to foreground.

```
akansha@localhost:~$ sleep 100 &
[1] 3712
akansha@localhost:~$ sleep 200 &
[2] 3713
akansha@localhost:~$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
akansha@localhost:~$ fg %1
sleep 100
```

26. Use pkill with a safe target process name to terminate all active instances of that command simultaneously.

```
akansha@localhost:~$ pkill -f sleep
[2]+  Terminated              sleep 200 &
```

27. Install the tree package using dnf, and run tree on your home directory.

```
akansha@localhost:~$ sudo dnf install -y tree
Complete!
akansha@localhost:~$ tree ~ | head -n 8
/home/akansha
├── document.txt
├── notes.txt
└── script.sh
```

28. Remove the tree package after testing, and verify its uninstallation using the package manager list command.

```
akansha@localhost:~$ sudo dnf remove -y tree
Removed!
akansha@localhost:~$ dnf list installed tree
Error: No matching Packages to list
```

29. Check your default umask setting, create a new file named test_mask.txt, and analyze how umask influenced permissions.

```
akansha@localhost:~$ umask
0022
akansha@localhost:~$ touch test_mask.txt
akansha@localhost:~$ ls -l test_mask.txt
-rw-r--r--. 1 akansha akansha 0 Oct 31 10:30 test_mask.txt
Default base permissions for files are 666 (rw-rw-rw-). Applying umask 022
subtracts group/other write rights, resulting in 644 (rw-r--r--).
```

30. Filter system processes to list only those belonging to your logged-in username using ps aux | grep $USER.

```
akansha@localhost:~$ ps aux | grep $USER
akansha     3241  0.0  0.2 225412  5120 pts/0    S    10:15   0:00 bash
akansha     3780  0.0  0.1 222100  2400 pts/0    R+   10:31   0:00 grep --
color=auto akansha
```

31. Change group ownership of a directory named project_dir to group devteam using the chgrp command with sudo.

```
akansha@localhost:~$ mkdir project_dir
akansha@localhost:~$ sudo groupadd devteam
akansha@localhost:~$ sudo chgrp devteam project_dir
akansha@localhost:~$ ls -ld project_dir
drwxr-xr-x. 2 akansha devteam 6 Oct 31 10:32 project_dir
```

32. Launch a process with a lower scheduling priority using nice -n 10 sleep 300 & and confirm its priority using ps.

```
akansha@localhost:~$ nice -n 10 sleep 300 &
[1] 3815
akansha@localhost:~$ ps -o pid,ni,cmd -p 3815
  PID  NI CMD
 3815  10 sleep 300
```

33. Modify the priority of an already running process using the renice command by specifying its PID.

```
akansha@localhost:~$ renice -n 15 -p 3815
3815 (process ID) old priority 10, new priority 15
```

34. Search for the httpd (Apache) package, install it, and check its system status using systemctl status.

```
akansha@localhost:~$ sudo dnf install -y httpd
akansha@localhost:~$ systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor
preset: disabled)
     Active: inactive (dead)
```

35. Explain why executing apt/dnf install package without update/check-update first might cause installation failure.

```
Explanation: If repository metadata cache is outdated or missing, package URLs/
versions might fail to resolve properly, resulting in HTTP 404 or missing
dependency errors.
```

36. Create a shell script, grant execute permissions, execute it with ./script.sh, and explain shebang behavior.

```
akansha@localhost:~$ echo 'echo "Hello RHEL"' > script.sh
akansha@localhost:~$ chmod +x script.sh
akansha@localhost:~$ ./script.sh
Hello RHEL
Without #!/bin/bash, the current executing shell attempts to interpret script
binary/syntax directly, which can fail if syntax or environment differs.
```

37. Set permissions for shared.txt: owner (rw-), group (r--), others (---) — write both symbolic and numeric commands.

```
akansha@localhost:~$ touch shared.txt
akansha@localhost:~$ chmod u=rw,g=r,o= shared.txt
akansha@localhost:~$ chmod 640 shared.txt
akansha@localhost:~$ ls -l shared.txt
-rw-r-----. 1 akansha akansha 0 Oct 31 10:40 shared.txt
```

38. Start background task with nice -n 15 sleep 300 &, locate PID in top, and explain how 'nice value' impacts CPU time.

```
akansha@localhost:~$ nice -n 15 sleep 300 &
[1] 3910
A higher nice value (+15) means the process is "nicer" to other processes,
yielding CPU time to lower-nice or normal tasks when system resource contention
occurs.
```

39. Debug Scenario: A student runs kill 1234 but process refuses to terminate. What escalating command should be used next?

```
akansha@localhost:~$ kill -9 1234
kill -9 sends SIGKILL, forcing immediate kernel-level process destruction. It is
a last resort because the application cannot perform cleanup or release
resources safely.
```

40. Display top 5 CPU-consuming processes currently active on system using ps aux --sort=-%cpu | head -5.

```
akansha@localhost:~$ ps aux --sort=-%cpu | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.5 168432 13450 ?        Ss   08:11   0:01 /usr/lib/
systemd/systemd
akansha     3241  0.0  0.2 225412  5120 pts/0    Ss   10:15   0:00 -bash
root           2  0.0  0.0      0     0 ?        S    08:11   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   08:11   0:00 [rcu_gp]
```

41. Construct a single-line command using find and chmod to grant execute permissions (+x) to all .sh files in your working directory.

```
akansha@localhost:~$ find . -type f -name "*.sh" -exec chmod +x {} +
```

42. Explain the security risks associated with running chmod 777 on a sensitive system configuration file in production.

```
Explanation: chmod 777 grants Read, Write, and Execute access to ANY
unprivileged user on the system, allowing malicious actors or compromised
scripts to edit, wipe, or compromise critical configurations.
```

43. Install a package (e.g., tree), and use rpm -ql <pkg> to list every file installed onto the filesystem by that package.

```
akansha@localhost:~$ rpm -ql tree
/usr/bin/tree
/usr/share/doc/tree
/usr/share/man/man1/tree.1.gz
```

44. Explain the difference between process termination via SIGTERM (15) vs SIGKILL (9).

```
Explanation:
- SIGTERM (Signal 15): Asks the process to gracefully shut down, allowing it to close open file handlers and clear temporary memory.
- SIGKILL (Signal 9): Immediately terminates process execution at the kernel level without cleanup.
```

45. Create two files owned by different users, then write a single find command that filters and lists files owned by a specific user.

```
akansha@localhost:~$ find . -user akansha
./notes.txt
./script.sh
./document.txt
```

46. Set the Set User ID (SUID) bit on an executable binary custom_tool, write chmod command, and explain implications.

```
akansha@localhost:~$ chmod u+s custom_tool
SUID causes the program to execute with the effective privileges of the file's owner rather than the user running it.
```

47. Set SGID bit on a shared folder /shared_dir using octal mode (2775), and explain group ownership inheritance.

```
akansha@localhost:~$ sudo chmod 2775 /shared_dir
SGID on directories ensures all newly created files/subfolders automatically inherit the group ownership of the directory rather than the primary group of the creating user.
```

48. Set Sticky Bit on a directory public_tmp (chmod 1777 or o+t), and explain file deletion protection.

```
akansha@localhost:~$ chmod 1777 public_tmp
Sticky bit allows all users to create files in the folder, but users can only delete or rename files that they own (or if they are root).
```

49. Demonstrate job management: Suspend a command (Ctrl+Z), verify with jobs, resume in background (bg), attach in foreground (fg).

```
akansha@localhost:~$ sleep 500
^Z
[1]+  Stopped                 sleep 500
akansha@localhost:~$ bg
[1]+ sleep 500 &
akansha@localhost:~$ fg
sleep 500
```

50. Write command pipeline that identifies top memory consuming process, logs details to top_memory.log, and terminates it gracefully.

```
akansha@localhost:~$ PID=$(ps aux --sort=-%mem | awk 'NR==2 {print $2}'); ps -p $PID -o pid,ppid,cmd,%mem > top_memory.log; kill -15 $PID
```